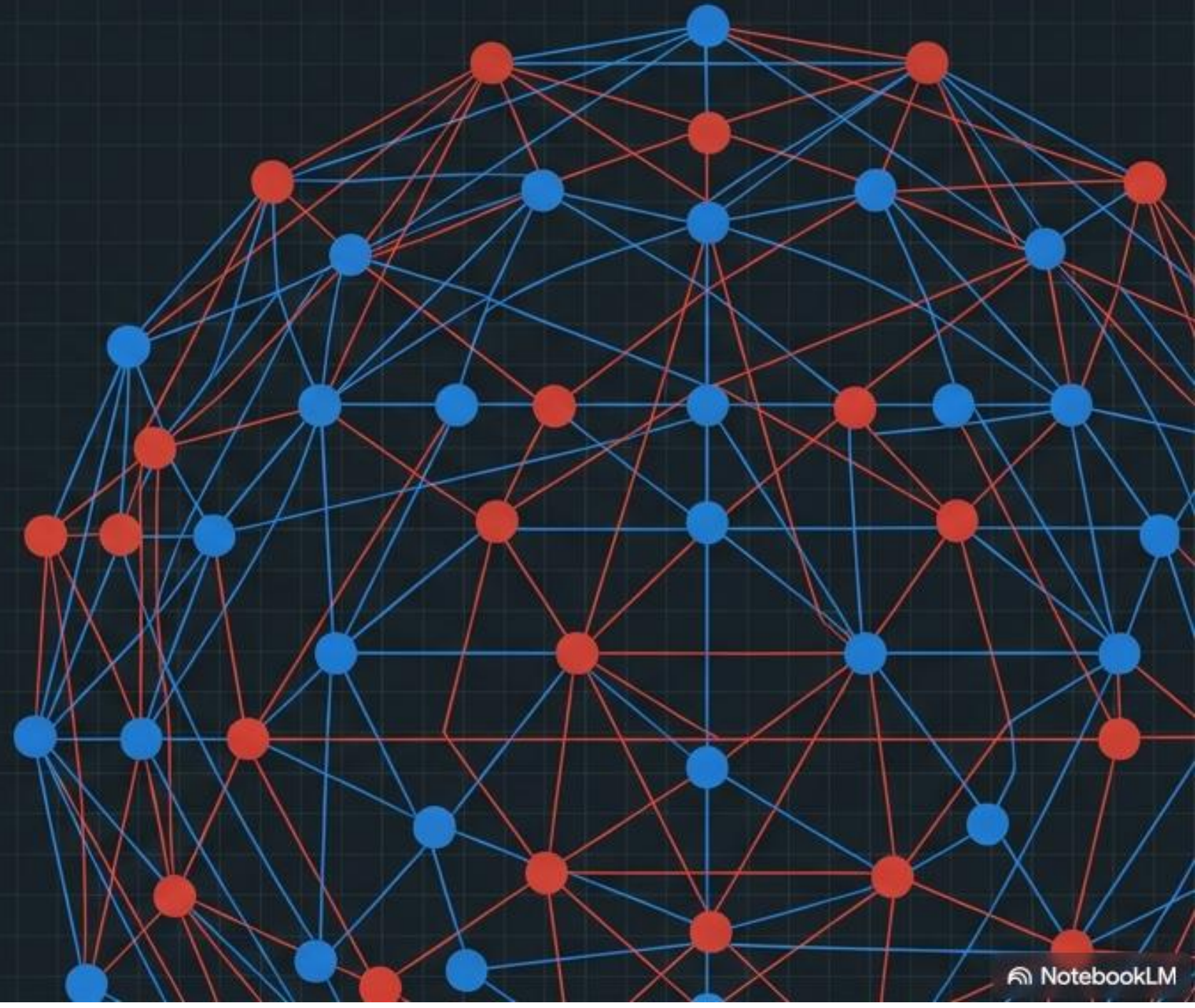


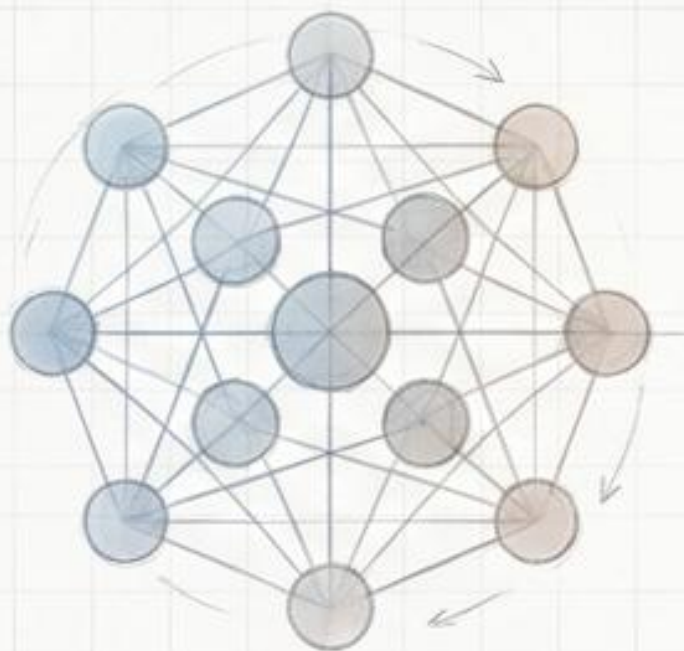
Graflowe Bazy Danych

Budowa, zapytania
i analiza grafów wiedzy
w praktyce

dr Zbigniew Galar



Przeszłość



Teraźniejszość



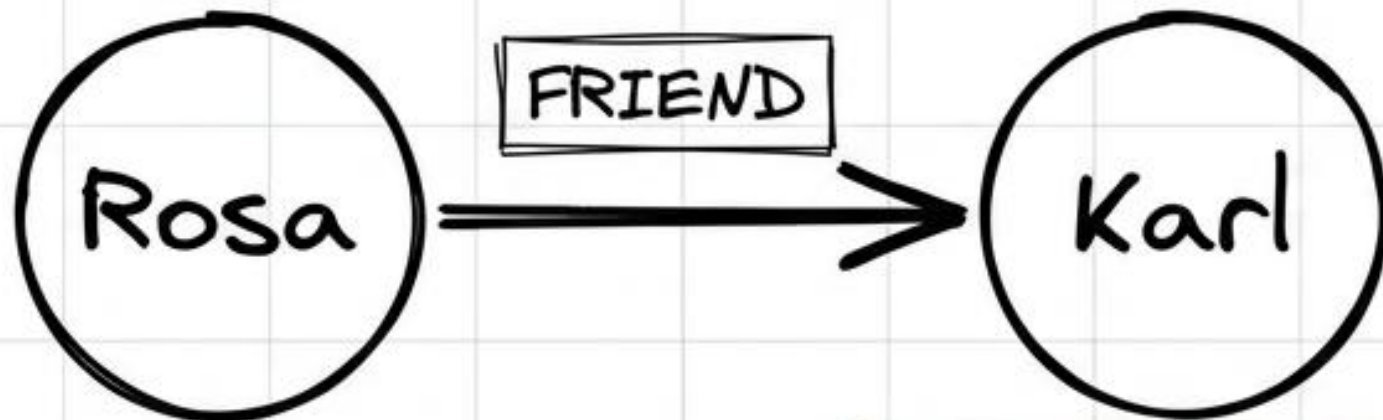
Środowisko akademickie i wczesny
Semantic Web. Koncepty wyprzedzające
możliwości technologiczne.

Skalowalne bazy grafowe (Neo4j),
Graph Data Science, chmura (AuraDB).

Nowoczesna technologia sprawiła, że grafy wiedzy nie są już tylko teorią – stały się potężnym silnikiem napędzającym dzisiejsze aplikacje biznesowe.

Filozofia Cypher: Intuicja kontra Kod

Tablica / Intuicja



Rysujesz to, co zapisujesz.

Terminal / Kod

> █

```
(Rosa)-[:FRIEND]->(Karl)
```

Cypher to deklaratywny język zapytań (standaryzowany jako GQL). Został zaprojektowany tak, aby struktura zapytania wizualnie przypominała strukturę samego grafu wiedzy.



Węzły (Nodes)

Encje w grafie.



Relacje (Relationships)

Zawsze skierowane, o stałym koszcie przejścia (brak spowolnień znanych z relacyjnych operacji JOIN).

(:Person {name:'Rosa'})-[:LIVES_IN]->(:Place)



Etykiety (Labels)

Role węzłów i indeksy bazy.



Właściwości (Properties)

Pary klucz-wartość przechowujące dane (np. name: 'Rosa').

Oczekiwanie / Kod

```
CREATE (:Person {name: 'Rosa'})-[:LIVES_IN]->(:Place {city: 'Berlin'})
```

Błędne założenie:
Baza wie, że Rosa i Berlin już istnieją.

Rzeczywistość



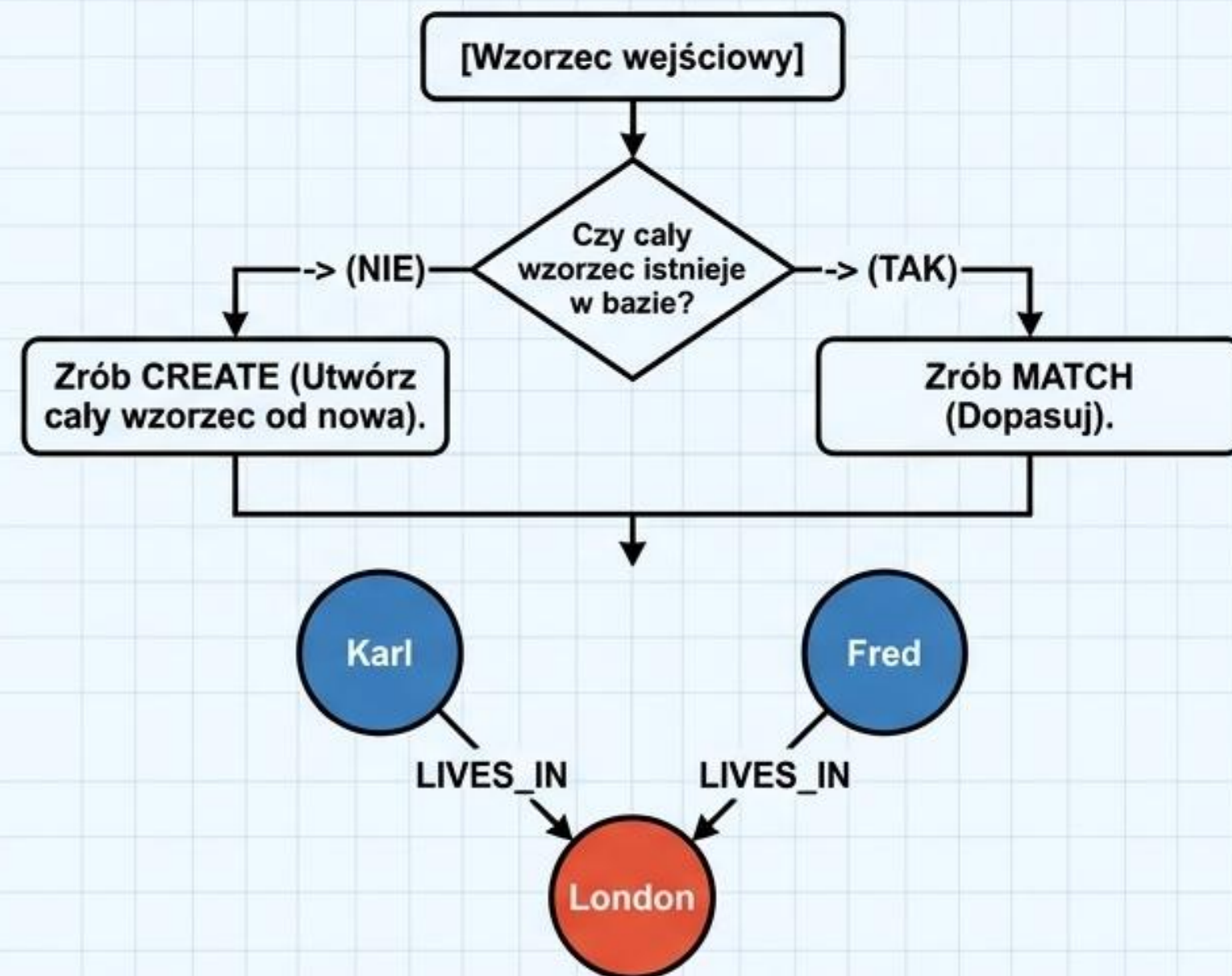
Pułapka CREATE: Polecenie **CREATE** zawsze tworzy nowe dane. Uruchomione dwukrotnie, stworzy zduplikowane, niezależne węzły zamiast je połączyć.

Krok 1: Twarde Reguły (Constraints)

```
CREATE CONSTRAINT ...  
  REQUIRE (p.country, p.city) IS NODE KEY
```



Krok 2: Logika MERGE (Schemat Decyzyjny)



Zasada: MERGE to operacja atomowa. Nigdy nie łączy częściowego dopasowania. Wymaga rozbicia na osobne linie dla węzłów i relacji.

Cykl Życia Danych w Cypher

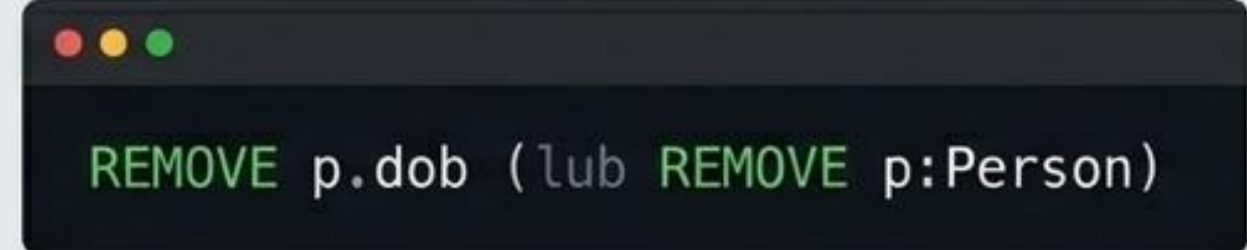
Aktualizacja (SET)



```
SET p.dob = 19841203
```

Dodawanie i modyfikacja właściwości w locie.

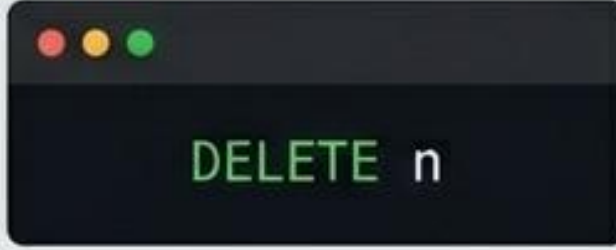
Czyszczenie (REMOVE)



```
REMOVE p.dob (lub REMOVE p:Person)
```

Bezpieczne usuwanie właściwości lub całych etykiet z węzła.

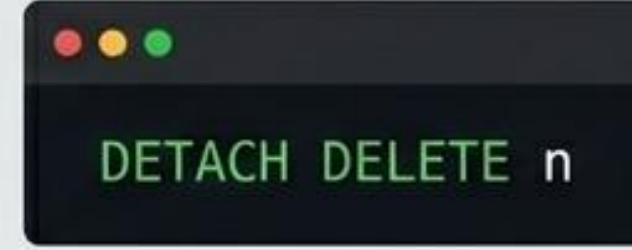
Zwykłe Usuwanie (DELETE)



```
DELETE n
```

Usuwa węzeł, ale przerywa transakcję, jeśli węzeł ma wciąż aktywne relacje. Chroni przed 'wiszącymi' relacjami.

Twardy Reset (DETACH DELETE)



```
DETACH DELETE n
```

Brutalna siła. Kaskadowo usuwa węzeł oraz wszystkie połączone z nim relacje.

Zapytania Lokalne: Kotwiczenie w Grafie



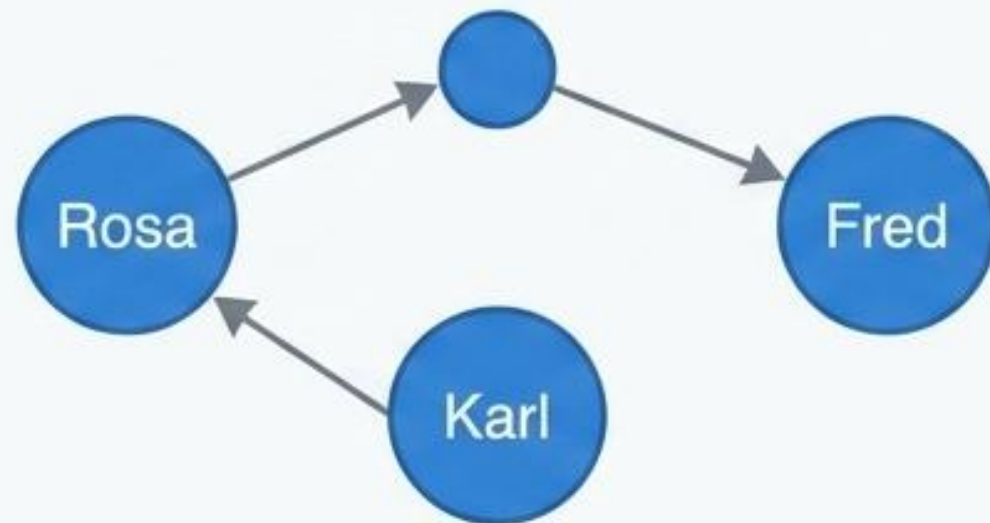
```
MATCH (p:Person)-[:LIVES_IN]->(Place {city:'Berlin'})  
RETURN p
```

- Zasada: Wiążemy zapytanie ze znanym punktem początkowym (np. Berlin).
- Mechanika: Baza danych przeszukuje tylko **lokale otoczenie** węzła, ignorując resztę grafu. Czas zapytania zależy od wielkości przeszukiwanego podgrafu, a nie od wielkości całej bazy.

Eksploracja Głębiniowa i Unikanie Cykli

Problem

```
–[:FRIEND*2..2]–>(fof:Person)
```



Domyślna ścieżka pozwala na powroty. Rosa staje się przyjacielem swojego przyjaciela poprzez Karla.

Rozwiązanie

```
WHERE rosa <> fof
```



Klauzula WHERE eliminuje nielogiczne cykle.

Fakt inżynieryjny: W przeciwieństwie do baz SQL, głębokie trawersowanie grafu nie wywołuje problemu 'join-bomb'. Koszt przejścia relacji to mikrosekundy.

Zapytania Globalne: Analiza Przekrojowa

```
MATCH (p:Place)<-[l:LIVES_IN]-(:Person)
RETURN p.city, count(l) AS rels ORDER BY rels DESC
```

place	rels
{"country":"UK","city":"London"}	2
{"country":"DE","city":"Berlin"}	1

Kluczowa Zasada Optymalizacji

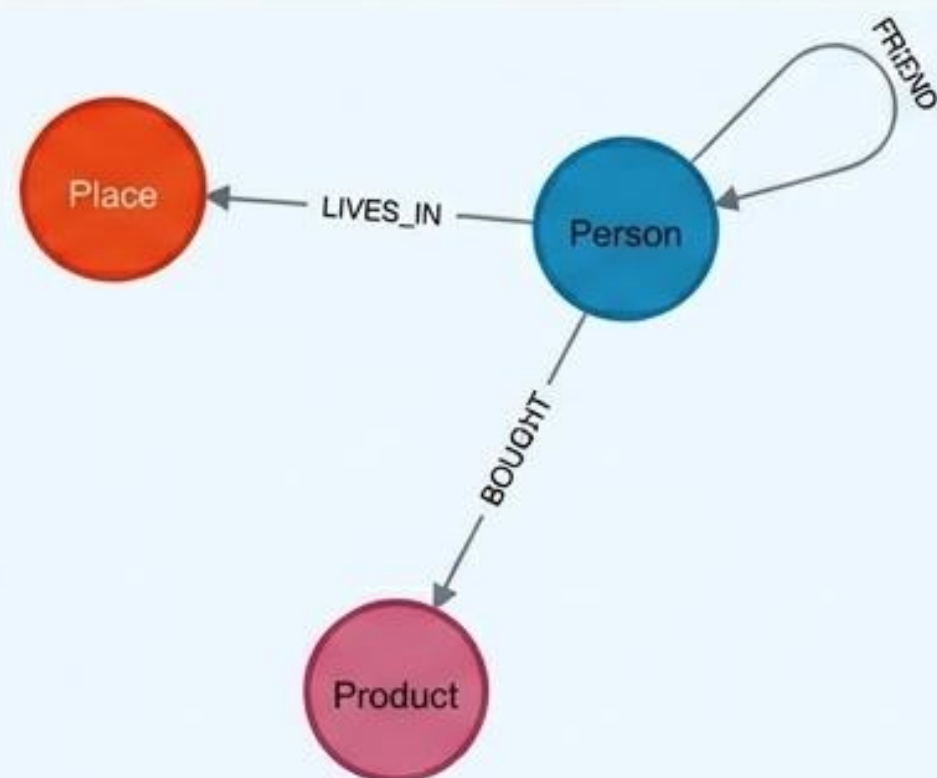
Zapytania globalne nie mają kotwicy. Skanują całą populację określonych węzłów.

Zawsze używaj etykiet. Pominięcie **etykiety (:Person)** na rzecz anonimowego węzła **()** drastycznie powiększa przestrzeń poszukiwań i zabija wydajność.

Gotowe Narzędzia: Procedury i Funkcje

Procedury Wewnętrzne

```
CALL db.schema.visualization()
```



Wbudowane procedury do inspekcji bazy i zarządzania architekturą.

Biblioteka APOC

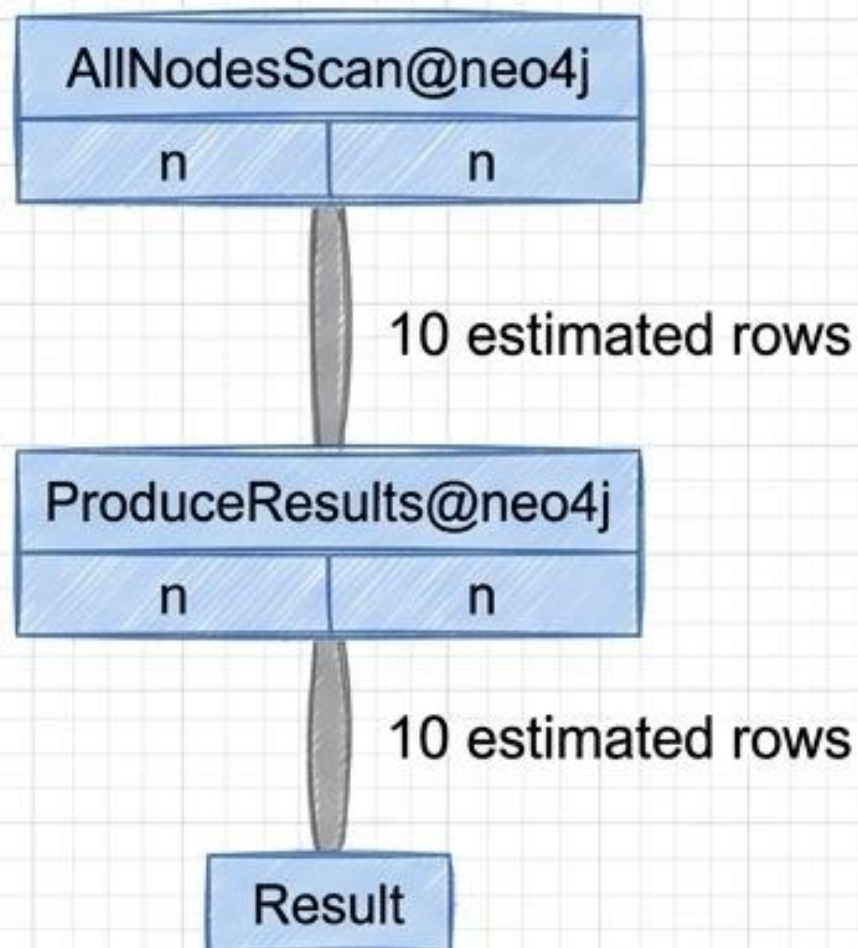
```
CALL apoc.atomic.concat(p, 'name', 'dy')
```



Awesome Procedures On Cypher. Potężny zestaw setek funkcji do manipulacji danymi, transformacji dat i zaawansowanych algorytmów grafowych bez pisania własnego kodu.

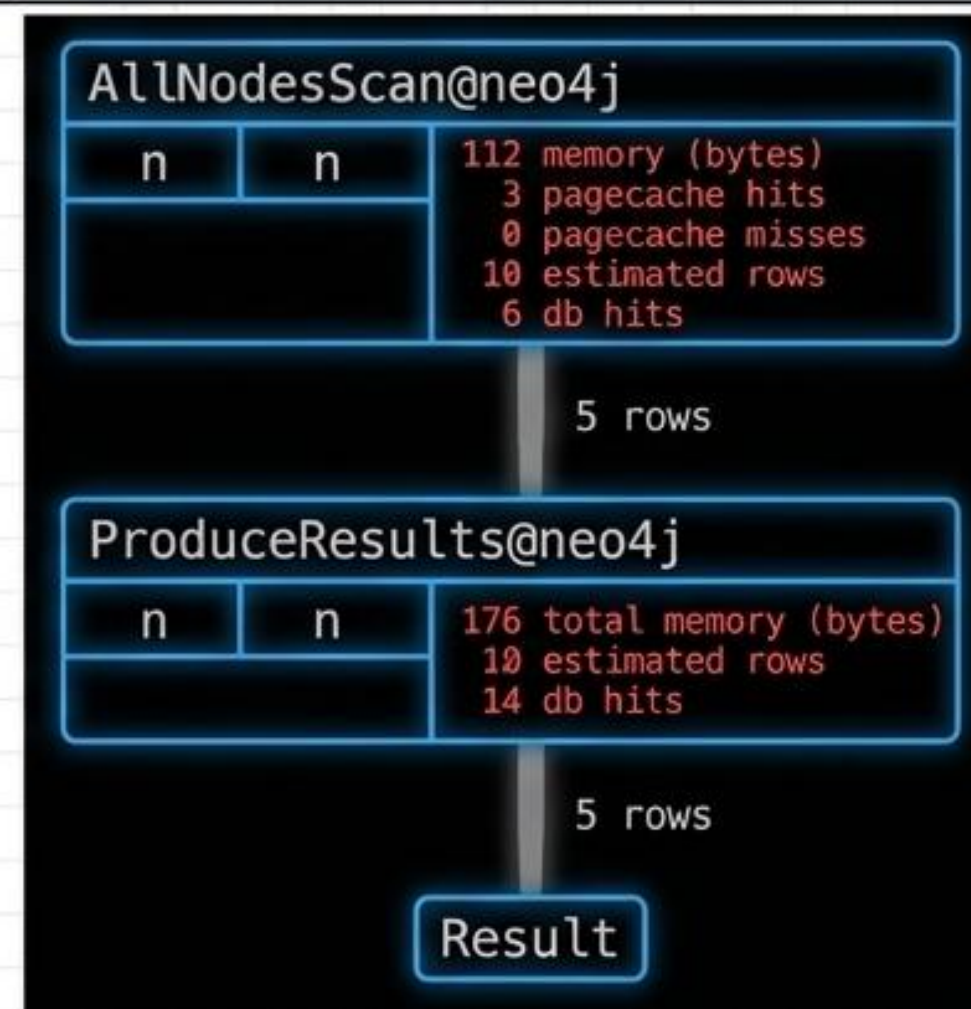
Profilowanie i Optymalizacja Zapytań

EXPLAIN



Symulacja. Pokazuje, jak planer zapytań zamierza wykonać kod. Szacuje liczbę wierszy bez faktycznego dotykania bazy danych. Idealne do bezpiecznego testowania.

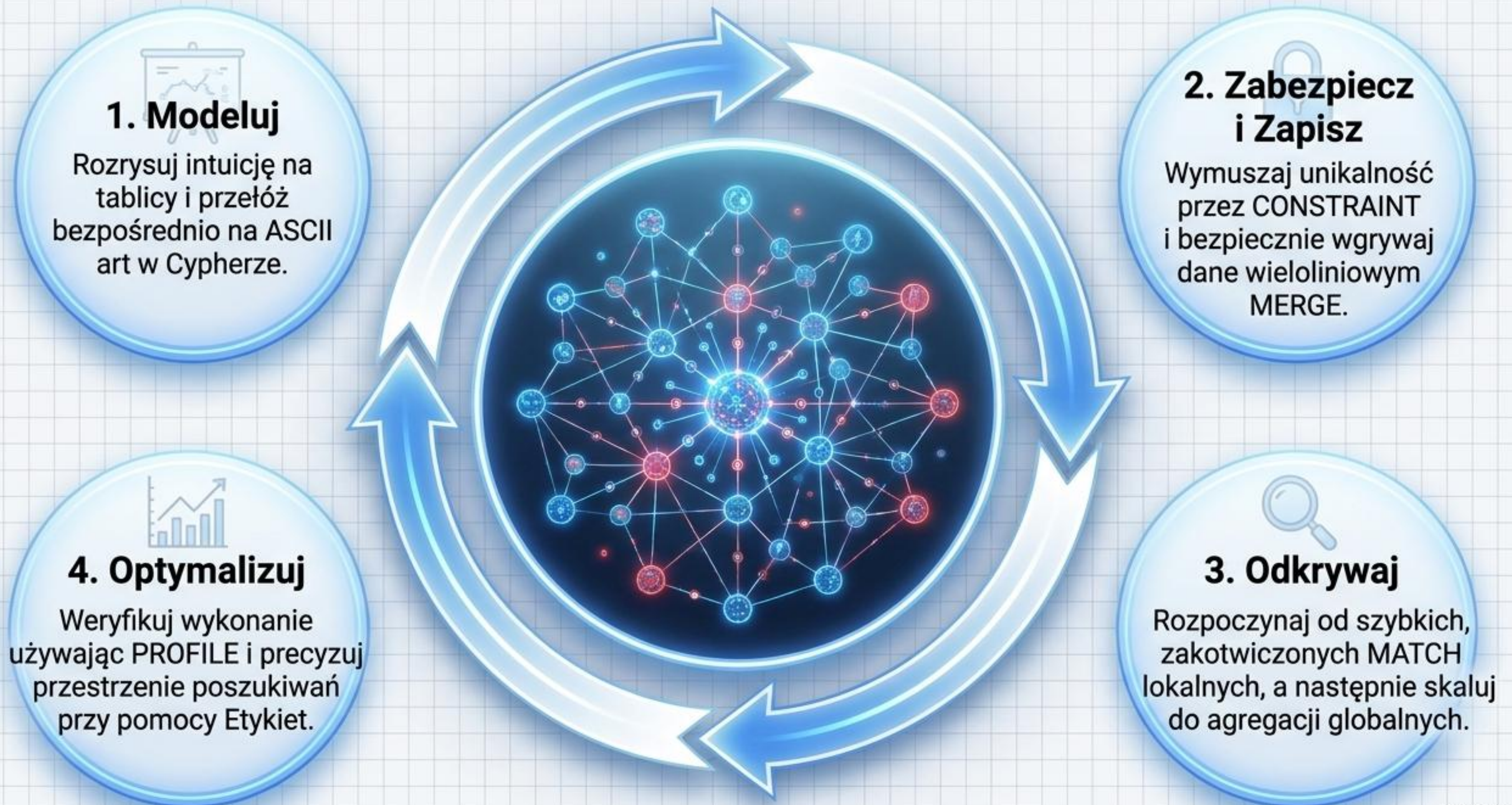
PROFILE



Rzeczywistość. Wykonuje zapytanie i śledzi prawdziwe statystyki wykonania.

Cel Optymalizacji: Minimalizuj DB hits (uderzenia w bazę). Filtruj dane (kardynalność) tak wcześnie, jak to możliwe w strukturze zapytania.

Kompletny Cykl Pracy z Grafem



Od teorii do Twojego własnego grafu.

Lokalnie

Pobierz aplikację Neo4j Desktop – pełne środowisko analityczne do eksperymentów.

W Chmurze

Użyj Neo4j AuraDB (darmowy tier) – błyskawiczny start i prototypowanie systemów produkcyjnych.